Robustness and Stability Analysis Report

Risk-Prioritization Security Assessment Model

Table of Contents

1. INTRODUCTION 2

2. OVERVIEW OF THE MATHEMATICAL MODEL 5

2.1 Risk Index and Objective Weights (ORI) 5
2.2 Topic effectiveness aggregation 5
2.3 Interdependency propagation 5
2.4 Final scoring and ranking 5

3. DEFINITION OF TESTS AND RATIONALE 6

3.1 Monte Carlo Uncertainty Analysis 6
Purpose 6
How it works 6
Interpretation of results 6
3.2 Rank Correlation Metrics (Spearman and Kendall) 6
Purpose 6
How it works 6
Interpretation of results 7
3.3 Pairwise Flip Rate 7
Purpose 7
How it works 7
Interpretation of results 7
3.4 Topic-Level Rank Change and Displacement 7
Purpose 7
How it works 7
Interpretation of results 8
3.5 Stress Testing on Objective Weights 8
Purpose 8
How it works 8
Interpretation of results 8
3.6 Structural Limit Tests (A and B) 8
Purpose 8
How it works 8
Interpretation of results 8
3.7 Objective Dominance Tests 9
Purpose 9
How it works 9
Interpretation of results 9
3.8 Numerical Conditioning Check 9
Purpose 9

How it works 9
Interpretation of results 9
3.9 Noise-Model Comparison 9
Purpose 9
How it works 9
Interpretation of results 10

4. SIMULATION & ANALYSIS METHODOLOGY 10

5. RESULTS AND INTERPRETATION 11

5.1 Baseline ranking 11
5.2 Monte Carlo robustness results 11
5.3 Topic-level stability 11
5.4 Score dispersion 12
5.5 Stress tests on objective weights 13
5.6 Structural limit tests 14
5.7 Objective dominance tests 14
5.8 Numerical conditioning 15
5.9 Noise-model comparison 15

6. CONCLUSION 16

REFERENCES: 16

1. Introduction

This report summarizes the robustness, stability, and structural coherence analyses performed on the Risk-Prioritization security assessment model, a mathematical framework designed to prioritize cybersecurity domains according to their contribution to organizational risk reduction. The model integrates multi-objective effectiveness assessments, objective importance weights derived from a formal risk index, and interdependencies between cybersecurity domains. The analysis was performed on the North-Western Campus assessment results.

All parameters used in the assesment are estimated quantities, not directly observed empirical measurements. Consequently, the objective of this report is not to validate empirical accuracy, but to evaluate the robustness, reliability, and numerical soundness of the resulting rankings under plausible uncertainty.

Specifically, the analyses aim to answer the following questions:
- How stable are the rankings under uncertainty in model parameters?
- Are ranking variations localized or structural?
- Does any single objective or modeling choice dominate the outcome?
- Does the model behave correctly under theoretical limit conditions?

This report claims that the ranking is robust within the modeled uncertainty space, the model is numerically stable and structurally coherent, and ranking variability is bounded and interpretable. It does not claim that the ranking represents an objective or absolute truth, that the model is empirically validated against real-world outcomes, or that uncertainty has no effect on prioritization. Results should therefore be interpreted as robust decision support, not deterministic optimization.

2. Overview of the Mathematical Model

2.1 Risk Index and Objective Weights (ORI)

Prioritization begins by quantifying the 'Objective Risk Index" (ORI) associated with each high level objective of an organization:

$$R_o^{(\sigma)} = \lambda_o \cdot \iota_o \cdot \epsilon_o \cdot a_o \cdot \ell_o^{(\sigma)}$$

Objective weights are obtained through normalization:

$$w_o^{(\sigma)} = \frac{R_o^{(\sigma)}}{\sum_{k=1}^{p} R_k^{(\sigma)}}$$

2.2 Topic effectiveness aggregation

We define the per-objective effectiveness of topic on objective by:

$$\eta_{ot} = \beta\, r_{ot} + (1-\beta)\, v_t\, u_{ot}$$

2.3 Interdependency propagation

Dependencies between topics are captured by an interdependency matrix D, scaled by parameter α:

$$\boldsymbol{\eta}'_o = (I + \alpha D)\, \boldsymbol{\eta}_o, \qquad \alpha \in [0,1]$$

2.4 Final scoring and ranking

Final topic scores are computed as:

$$\text{Score}(t) = \sum_{o=1}^{p} w_o^{(\sigma)}\, \eta'_{ot}.$$

3. Definition of Tests and Rationale

3.1 Monte Carlo Uncertainty Analysis

Purpose

Evaluate ranking stability under simultaneous uncertainty in all estimated parameters (effectiveness scores and ORI weights).

How it works

All estimated model parameters are randomly perturbed within a bounded uncertainty range (±10%), simulating plausible estimation errors.
For each simulation:

- ORI objective weights are perturbed and renormalized.
- Topic effectiveness parameters r, u, and v are perturbed.
- Topic scores and rankings are recomputed.

This process is repeated thousands of times to generate a distribution of possible rankings under uncertainty.

Interpretation of results

Result	Meaning
High correlation across simulations	Ranking is robust to estimation uncertainty
Large ranking variability	Model is sensitive to parameter estimation
Stable top and bottom topics	Strategic priorities are reliable
Frequent major reshuffling	Structural instability

3.2 Rank Correlation Metrics (Spearman and Kendall)

Purpose

Measure preservation of global and pairwise ordering under perturbation.

How it works

For each perturbed ranking:

- Spearman correlation measures global monotonic agreement with the baseline ranking.
- Kendall correlation measures pairwise ordering consistency.

Interpretation of results

Value range	Meaning
ρ or $\tau \approx 1$	Rankings nearly identical
ρ or $\tau \geq 0.9$	Strong stability
ρ or τ between 0.6 and 0.9	Moderate sensitivity
ρ or $\tau < 0.6$	Structural instability

3.3 Pairwise Flip Rate

Purpose

Quantify how often the relative ordering between topic pairs reverses.

How it works

For each pair of topics:

- Check whether their relative order changes between baseline and perturbed rankings.
- Compute the proportion of reversed pairs across simulations.

Interpretation of results

Flip rate	Meaning
Less than 5%	Very stable ranking
Between 5% and 15%	Moderate local variation
Greater than 20%	Structural instability

Low flip rates indicate localized variation rather than systemic reordering.

3.4 Topic-Level Rank Change and Displacement

Purpose

Distinguish minor local rank swaps from material ranking instability.

How it works

For each topic:

- Measure frequency of rank change.
- Compute expected rank displacement:

Expected displacement = average of |simulated rank − baseline rank|

Interpretation of results

Displacement	Meaning
Less than 1 rank	Localized variation
Between 1 and 2 ranks	Moderate sensitivity
Greater than 2 ranks	Structural instability

Localized displacement implies bounded uncertainty effects.

3.5 Stress Testing on Objective Weights

Purpose

Assess sensitivity to misestimation of ORI objective importance.

How it works

Each objective weight is independently stressed by ±10% while others remain unchanged and are then renormalized. Rankings are recomputed and compared to baseline.

Interpretation of results

Outcome	Meaning
Minor ranking change	Balanced multi-objective model
Strong ranking change	Dominant objective
Stable correlations	ORI uncertainty is manageable

3.6 Structural Limit Tests (α and β)

Purpose

Verify model coherence in theoretical limit regimes.

How it works

Evaluate ranking behavior when:

- β = 0 (indirect effectiveness only)
- β = 1 (direct effectiveness only)
- α = 0 (no interdependency propagation)

Interpretation of results

Outcome	Meaning
Moderate ranking change	Coherent model
Ranking collapse	Mechanism dominance
High correlation with baseline	Mechanisms are complementary

3.7 Objective Dominance Tests

Purpose

Determine whether the model collapses to a single objective.

How it works

Each objective is assigned full weight individually (w = 1), while all other objective weights are set to zero. Rankings are recomputed and compared to baseline.

Interpretation of results

Outcome	Meaning
Low similarity to baseline	Truly multi-objective
High similarity	Hidden objective dominance

3.8 Numerical Conditioning Check

Purpose

Ensure numerical stability and absence of noise amplification.

How it works

Compute the condition number of the interdependency matrix M.

Interpretation of results

Condition number	Meaning
Less than 10^3	Well-conditioned
Between 10^3 and 10^6	Moderate sensitivity
Greater than 10^6	Potential instability

3.9 Noise-Model Comparison

Purpose

Test robustness against different uncertainty distributions.

How it works

Repeat Monte Carlo simulations using:

- Uniform noise
- Truncated Gaussian noise

Compare ranking stability metrics between both simulations.

Interpretation of results

Outcome	Meaning
Similar results	Distribution-agnostic robustness
Divergent results	Assumption-dependent ranking

4. Simulation & Analysis Methodology

All robustness and stability results presented in this section were obtained through **systematic numerical simulations implemented in Python**. The analysis framework was developed to faithfully reflect the mathematical structure of the North-Western security assessment model and to ensure reproducibility and numerical rigor.
The simulations rely on:

- Monte Carlo sampling to propagate uncertainty across estimated model parameters,
- Repeated recomputation of effectiveness matrices, interdependency propagation, and final scores,
- Statistical evaluation of ranking stability using rank correlation metrics, flip rates, and rank displacement measures.

Each simulation run perturbs model inputs within predefined uncertainty bounds while preserving theoretical constraints such as normalization of objective weights and bounded effectiveness parameters. Thousands of independent simulation runs are executed to obtain statistically meaningful distributions of robustness indicators.
All computations, simulations, and visualizations were performed using **Python**, leveraging standard scientific libraries for numerical computation, data manipulation, and statistical analysis.

The code is presented in Annexe 1.

5. Results and Interpretation

5.1 Baseline ranking

Topic	Score	Rank
Critical Infrastructure Security	0.830498	1
Security Operations	0.747943	2
Identity and Access Management (IAM)	0.747128	3
Supply Chain Security	0.711088	4
Network Security	0.666503	5
Risk analysis	0.612607	6
Physical Security	0.574052	7
Security Awareness & Training	0.572405	8
Cyber Threat Intelligence	0.539539	9
Cryptography	0.376258	10

Table 1 – Baseline topic ranking and scores

5.2 Monte Carlo robustness results

Interpretation: The ranking exhibits very high global and pairwise stability under uncertainty. Ranking changes are infrequent and limited in magnitude:

```
Spearman mean: 0.9861430303030304
Spearman min : 0.9393939393939393
Kendall mean : 0.9517200000000001
Kendall min  : 0.8666666666666666
Flip rate avg: 0.024139999999999998
```

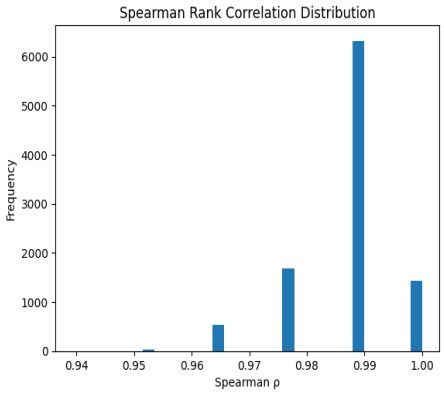

Figure 1– Spearman correlation distribution

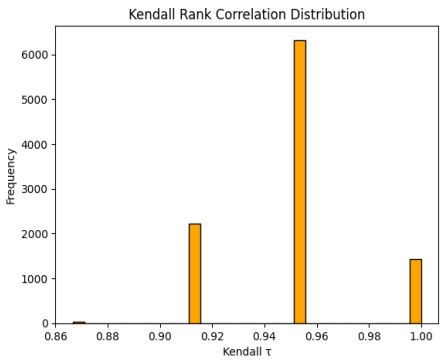

Figure 2– Kendall correlation distribution

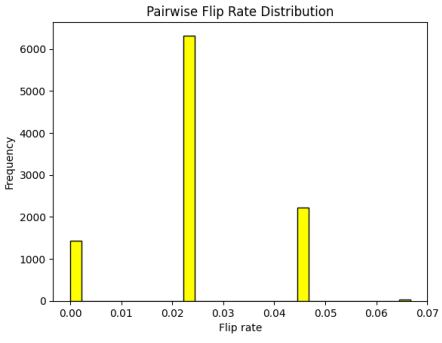

Figure 3– Pairwise flip-rate distribution

5.3 Topic-level stability

Interpretation: Top-ranked and bottom-ranked topics are highly stable. Rank variability is concentrated among mid-ranked topics and remains bounded.

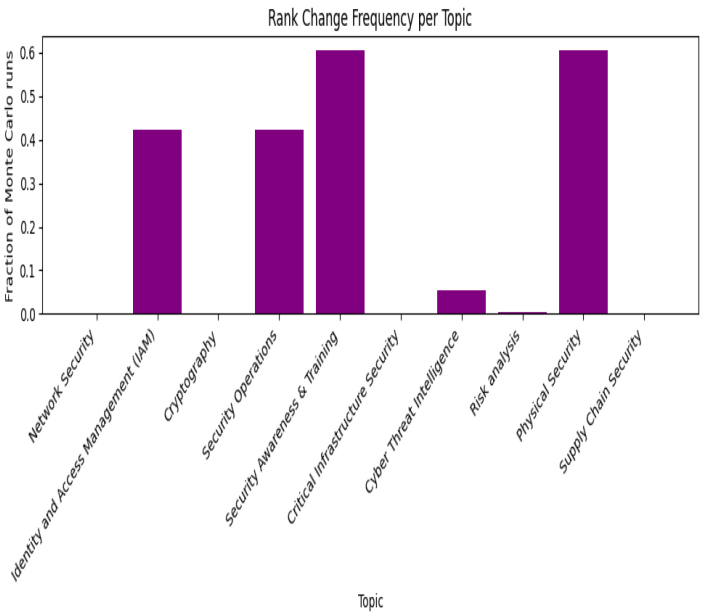

Figure 4– Rank change frequency per topic

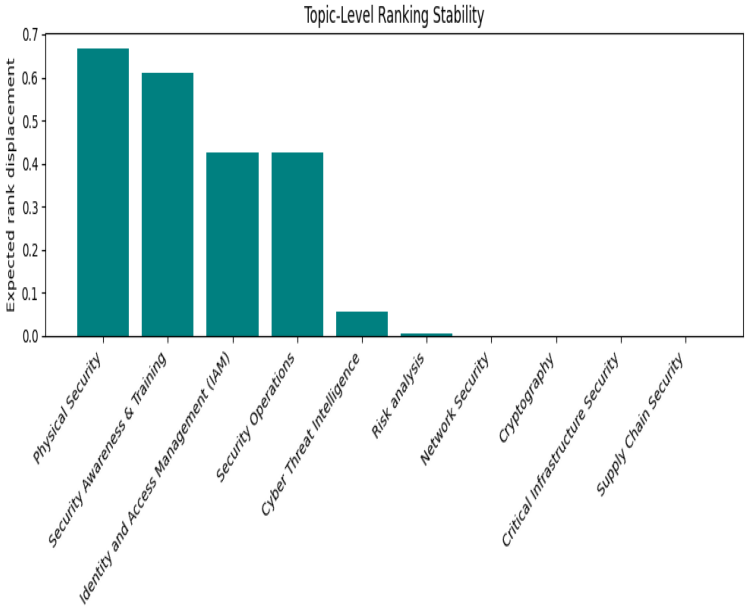

Figure 5– Expected rank displacement per topic

5.4 Score dispersion

Interpretation: Higher score dispersion correlates with rank mobility but does not indicate numerical instability.

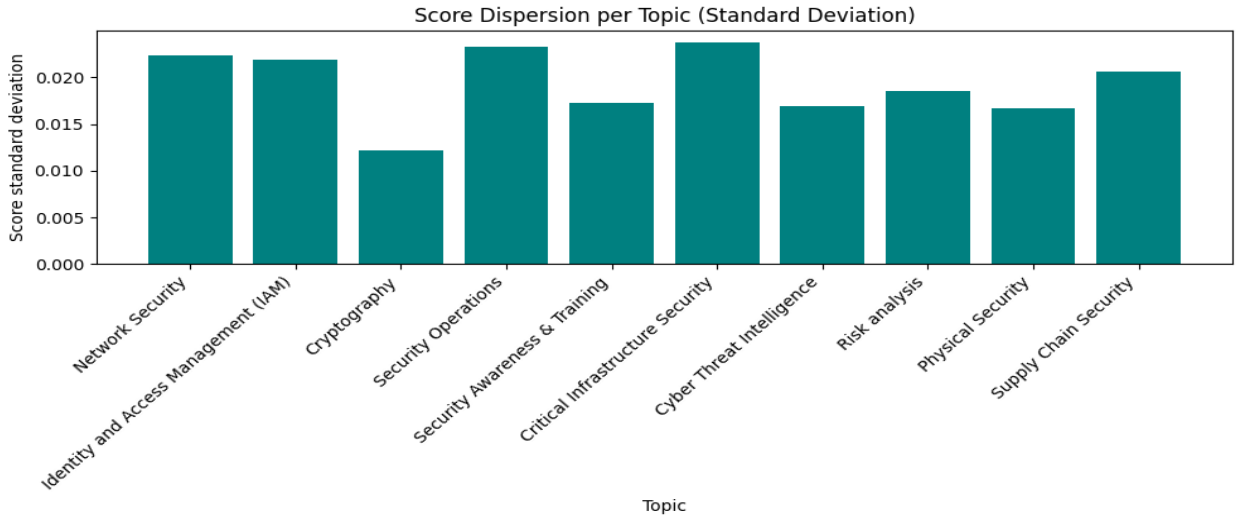

Figure 6 – Score standard deviation per topic

5.5 Stress tests on objective weights

Scenario	Spearman	Kendall	Flip rate
Av -10%	0.987879	0.955556	0.022222
Av +10%	0.987879	0.955556	0.022222
Phy -10%	0.987879	0.955556	0.022222
Phy +10%	0.987879	0.955556	0.022222
AM -10%	1.000000	1.000000	0.000000
AM +10%	0.987879	0.955556	0.022222
CIA -10%	1.000000	1.000000	0.000000
CIA +10%	0.975758	0.911111	0.044444

Table 2 – Stress test correlations and flip rates

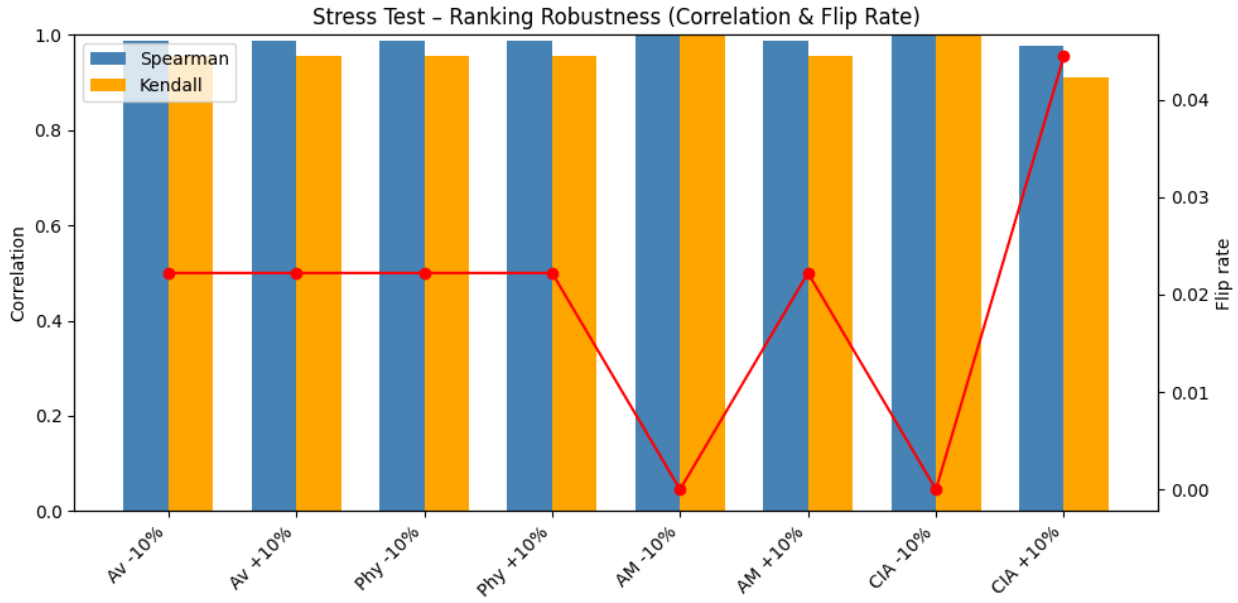

Figure 7 – Stress test robustness summary

5.6 Structural limit tests

Interpretation: The model behaves coherently under all theoretical limit conditions.

Scenario	Spearman	Kendall
$\beta = 0$	0.769697	0.644444
$\beta = 1$	0.866667	0.777778
$\alpha = 0$	0.951515	0.866667

Table 3 – Structural limit correlations

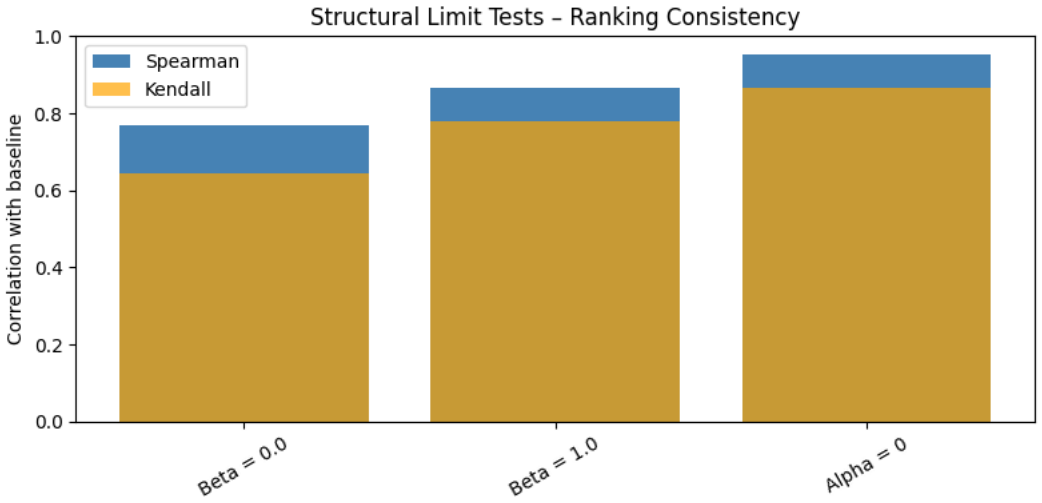

Figure 8 – Structural limit tests summary

5.7 Objective dominance tests

Interpretation: No objective alone reproduces the baseline ranking, confirming genuine multi-objective behavior.

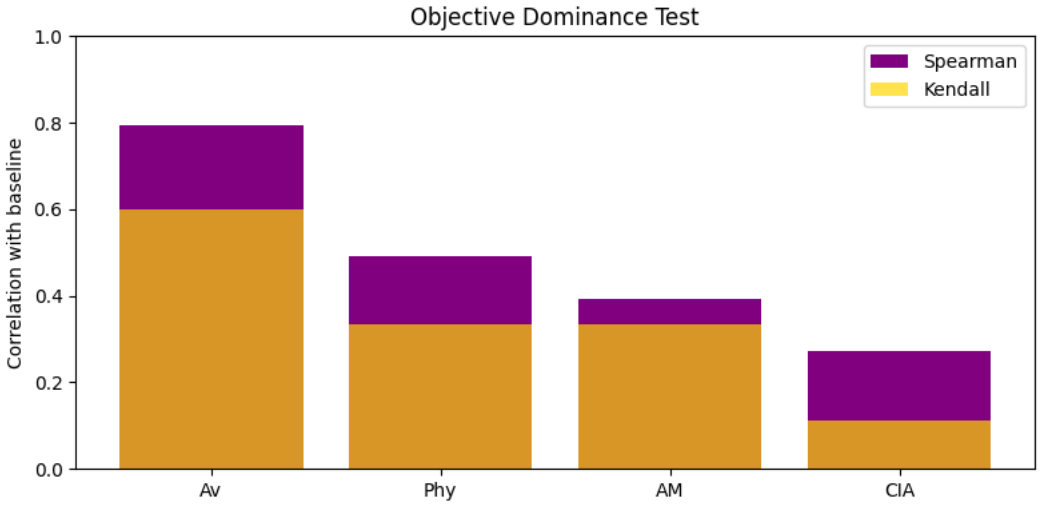

Figure 9 – Objective dominance correlations

5.8 Numerical conditioning

Interpretation: The model is numerically stable; robustness results are not artifacts of matrix conditioning.

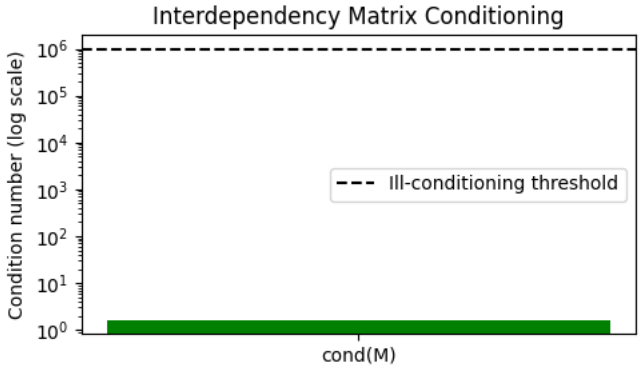

Figure 10 – Interdependency matrix condition number

5.9 Noise-model comparison

Interpretation: Robustness does not depend on the assumed uncertainty distribution.

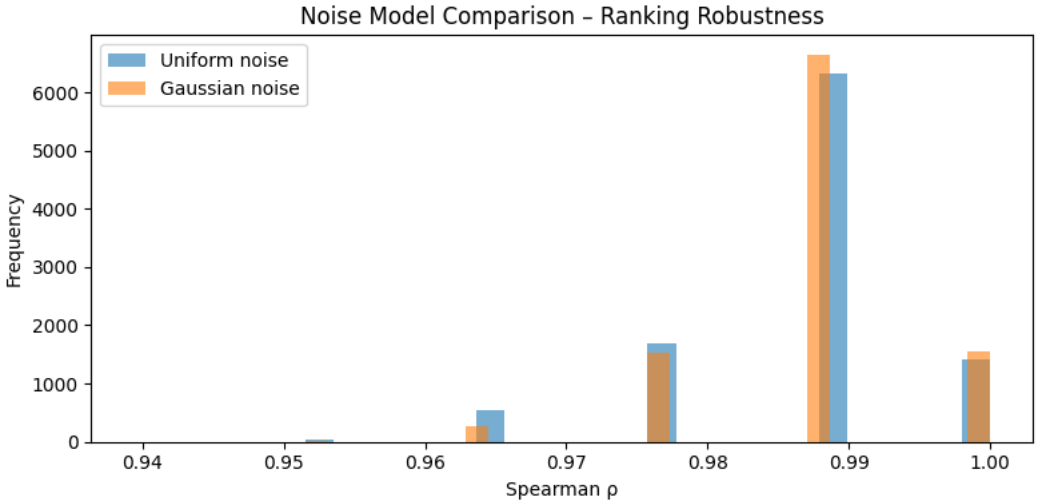

Figure 11 – Uniform vs Gaussian noise robustness comparison

6. Conclusion

This analysis shows that the North-Western security assessment model demonstrates high robustness, structural consistency**, and numerical stability** when subjected to comprehensive uncertainty propagation, stress testing, and structural validation.
The results indicate that:

- The overall ranking remains stable under realistic uncertainty affecting all estimated parameters.
- No individual objective, parameter, or modeling assumption exerts disproportionate influence on the outcome.
- Observed ranking variations are localized and remain quantitatively bounded.
- The model behaves coherently across theoretical limit regimes, confirming its structural soundness.

Taken together, these findings support the use of the model for strategic prioritization and decision support, with the recommendation that closely ranked topics be interpreted as **priority tiers** rather **than strict** ordinal **positions**.

References:

1. Ryan R., Stengel R.F. (1991) "A Monte Carlo Approach to the Analysis of Control System Robustness." *IEEE, pp.*
2. Mazurek J., Strzałka D. (2022) "On the Monte Carlo weights in multiple criteria decision analysis." *PLoS ONE, 17(10).*
3. Wicklin R. (2023) "Weak or strong? How to interpret a Spearman or Kendall correlation." *The DO Loop (SAS blog).*
4. Shah N.B., Wainwright M.J. (2018) "Simple, Robust and Optimal Ranking from Pairwise Comparisons." *JMLR, 18.*
5. Azzini I., Munda G. (2025) "Sensitivity and Robustness Analyses in Social Multi-Criteria Evaluation of Public Policies." *J. Multi-Criteria Decision Analysis.*
6. Fiveable (2025) "Condition Number and Numerical Stability." *Advanced Matrix Computations Guide.*
7. Llorente F., Martino L., Read J., Delgado D. (2025) "A survey of Monte Carlo methods for noisy and costly densities." *ArXiv report*

Annexe

```
# %% [markdown]
# # Imports & configuration
# %%
import pandas as pd
import numpy as np
from scipy.stats import spearmanr, kendalltau
import matplotlib.pyplot as plt
np.random.seed(42)
EXCEL_PATH = "North-Western.xlsx"
xls = pd.ExcelFile(EXCEL_PATH)
# %% [markdown]
# ## Load ORI (Risk Weights)
# %%
# Read the Risk Weights sheet
risk_df = pd.read_excel(xls, sheet_name="Risk Weights")
# Drop undefined rows
risk_df = risk_df.dropna(subset=["Objective", "Weight w (normalized)"])
# Build weights dictionary
weights = {row["Objective"]: row["Weight w (normalized)"] for _, row in risk_df.iterrows()}
# Keep only specified objectives and normalize
weights = {k: weights[k] for k in ["Av", "Phy", "AM", "CIA"]}
total = sum(weights.values())
weights = {k: v / total for k, v in weights.items()}
print("Objective weights (ORI):", weights)
# %% [markdown]
# ## Load Topic Effectiveness (r, u, v)
# %%
topic_df = pd.read_excel(xls, sheet_name="Topic Effectiveness", header=1)
# Show head of the table
topic_df.head()
# %% [markdown]
# ## Compute η matrix (topic × objective)
# %%
BETA = 0.7
def compute_eta_matrix(topic_df, beta):
    r_cols = ["r_Av", "r_Phy", "r_AM", "r_CIA"]
    u_cols = ["u_Av", "u_Phy", "u_AM", "u_CIA"]
    df = topic_df.set_index("Topic").copy()
    df[r_cols + u_cols + ["v_j"]] = df[r_cols + u_cols + ["v_j"]].astype(float)
    eta = pd.DataFrame(index=df.index)
    for r_col, u_col in zip(r_cols, u_cols):
        obj = r_col.replace("r_", "")
        eta[obj] = beta * df[r_col] + (1 - beta) * df["v_j"] * df[u_col]
    return eta
eta_matrix = compute_eta_matrix(topic_df, BETA)
# Align with r matrix indices
r_matrix = topic_df.set_index("Topic")[["r_Av", "r_Phy", "r_AM", "r_CIA"]]
r_matrix = r_matrix.rename(columns={"r_Av":"Av", "r_Phy":"Phy", "r_AM":"AM", "r_CIA":"CIA"})
r_matrix = r_matrix[~r_matrix.index.isna()]
r_matrix = r_matrix.loc[~r_matrix.index.duplicated()]
eta_matrix = eta_matrix.loc[r_matrix.index]
print(eta_matrix.head())
# %% [markdown]
# ## Load Interdependency matrix D and α
# %%
# Read raw Interdependency sheet
D_raw = pd.read_excel(xls, sheet_name="Interdependency", header=None)
# Extract numeric D block (10x10) from rows 1–10, cols 1–10
D = D_raw.iloc[1:11, 1:11].astype(float).values
# Extract alpha from the sheet
ALPHA = float(D_raw.loc[D_raw.iloc[:, 0] == "Alpha"].iloc[0, 1])
I = np.eye(D.shape[0])
M = I + ALPHA * D
print("Alpha:", ALPHA)
print("D shape:", D.shape)
# %% [markdown]
# ## Compute η′ (eta prime)
# %%
def compute_eta_prime(eta_matrix, M):
    eta_p = M.dot(eta_matrix.values)
    return pd.DataFrame(eta_p, index=eta_matrix.index, columns=eta_matrix.columns)
eta_prime = compute_eta_prime(eta_matrix, M)
eta_prime.head()
# %% [markdown]
# ## Compute final scores & baseline ranking
# %%
def compute_scores(weights, eta_prime):
    w = pd.Series(weights, dtype=float)
    scores = eta_prime.mul(w, axis=1).sum(axis=1)
```

```
    scores.name = "Score"
    return scores
baseline_scores = compute_scores(weights, eta_prime)
baseline_ranks = baseline_scores.rank(ascending=False, method="average")
baseline_scores.sort_values(ascending=False), baseline_ranks.sort_values()
# %% [markdown]
# ## Monte Carlo simulation
# %%
N_SIM = 10_000
UNCERTAINTY = 0.10
spearman_vals, kendall_vals, flip_rates = [], [], []
baseline_order = baseline_ranks.sort_values().index.tolist()
pairs = [(i, j) for i in range(len(baseline_order)) for j in range(i+1, len(baseline_order))]
# Initialize once before Monte Carlo loop
rank_change_counts = {topic: 0 for topic in baseline_ranks.index}
score_history = {topic: [] for topic in baseline_scores.index}



for _ in range(N_SIM):
    # Perturb ORI
    perturbed_weights = {k: v * (1 + np.random.uniform(-UNCERTAINTY, UNCERTAINTY)) for k, v in weights.items()}
    total = sum(perturbed_weights.values())
    perturbed_weights = {k: v / total for k, v in perturbed_weights.items()}
    # Perturb r, u, v
    perturbed_topic_df = topic_df.copy()
    for col in ["r_Av","r_Phy","r_AM","r_CIA","u_Av","u_Phy","u_AM","u_CIA","v_j"]:
        perturbed_topic_df[col] *= (1 + np.random.uniform(-UNCERTAINTY, UNCERTAINTY))
        perturbed_topic_df[col] = perturbed_topic_df[col].clip(0, 1)
    eta_mc = compute_eta_matrix(perturbed_topic_df, BETA)
    eta_mc = eta_mc.loc[eta_matrix.index]
    eta_prime_mc = compute_eta_prime(eta_mc, M)
    new_scores = compute_scores(perturbed_weights, eta_prime_mc)
    new_ranks = new_scores.rank(ascending=False)
    rho, _ = spearmanr(baseline_ranks, new_ranks)
    tau, _ = kendalltau(baseline_ranks, new_ranks)
    spearman_vals.append(rho)
    kendall_vals.append(tau)
    new_order = new_ranks.sort_values().index.tolist()
    # Track rank changes (topic-level)
    for topic in baseline_ranks.index:
        if new_ranks.loc[topic] != baseline_ranks.loc[topic]:
            rank_change_counts[topic] += 1
    # Track score dispersion
    for topic in new_scores.index:
        score_history[topic].append(new_scores.loc[topic])



    flips = sum(
        (new_order.index(baseline_order[i]) - new_order.index(baseline_order[j])) * (i - j) < 0
        for i, j in pairs
    )
    flip_rates.append(flips / len(pairs))

rank_change_freq = {
    topic: count / N_SIM
    for topic, count in rank_change_counts.items()
}
score_std = {
    topic: np.std(values)
    for topic, values in score_history.items()
}



# %% [markdown]
# ## Monte Carlo summary statistics
# %%
print("Spearman mean:", np.mean(spearman_vals))
print("Spearman min :", np.min(spearman_vals))
print("Kendall mean :", np.mean(kendall_vals))
print("Kendall min  :", np.min(kendall_vals))
print("Flip rate avg:", np.mean(flip_rates))
print("\nRank change frequency per topic:")
print("-" * 40)

# %%
for topic, freq in sorted(
    rank_change_freq.items(),
    key=lambda x: x[1],
    reverse=True
):
```

```
    print(f"{topic:40s} : {freq:.3f}")
print("\nScore dispersion per topic (standard deviation):")
print("-" * 40)
for topic, std in sorted(
    score_std.items(),
    key=lambda x: x[1],
    reverse=True
):
    print(f"{topic:40s} : {std:.5f}")
# %% [markdown]
# ## Spearman histogram
# %%
plt.hist(spearman_vals, bins=30)
plt.title("Spearman Rank Correlation Distribution")
plt.xlabel("Spearman ρ")
plt.ylabel("Frequency")
plt.show()
# %% [markdown]
# ## Kendall histogram
# %%
plt.hist(kendall_vals, bins=30, color="orange", edgecolor="black")
plt.title("Kendall Rank Correlation Distribution")
plt.xlabel("Kendall τ")
plt.ylabel("Frequency")
plt.show()

# %% [markdown]
# ## Flip-rate histogram
# %%
plt.hist(flip_rates, bins=30, color="yellow", edgecolor="black")
plt.title("Pairwise Flip Rate Distribution")
plt.xlabel("Flip rate")
plt.ylabel("Frequency")
plt.show()

# %% [markdown]
# ## Stress tests
# %%
stress_results = []
baseline_order = baseline_ranks.sort_values().index.tolist()
pairs = [(i, j) for i in range(len(baseline_order)) for j in range(i+1, len(baseline_order))]
for obj in weights:
    for sign in [-1, 1]:
        stressed_weights = weights.copy()
        stressed_weights[obj] *= (1 + sign * UNCERTAINTY)
        total = sum(stressed_weights.values())
        stressed_weights = {k: v / total for k, v in stressed_weights.items()}
        stressed_scores = compute_scores(stressed_weights, eta_prime)
        stressed_ranks = stressed_scores.rank(ascending=False)
        # Correlations
        rho, _ = spearmanr(baseline_ranks, stressed_ranks)
        tau, _ = kendalltau(baseline_ranks, stressed_ranks)
        # Flip rate
        stressed_order = stressed_ranks.sort_values().index.tolist()
        flips = sum(
            (stressed_order.index(baseline_order[i]) -
             stressed_order.index(baseline_order[j])) * (i - j) < 0
            for i, j in pairs
        )
        flip_rate = flips / len(pairs)
        stress_results.append({
            "Scenario": f"{obj} {'+' if sign>0 else '-'}{int(UNCERTAINTY*100)}%",
            "Spearman": rho,
            "Kendall": tau,
            "FlipRate": flip_rate
        })
stress_df = pd.DataFrame(stress_results)
stress_df

# %% [markdown]
# ## Rank Change Frequency per Topic
# %%
plt.figure(figsize=(10, 5))
plt.bar(
    rank_change_freq.keys(),
    rank_change_freq.values(),
    color="purple"
)
plt.title("Rank Change Frequency per Topic")
plt.ylabel("Fraction of Monte Carlo runs")
plt.xlabel("Topic")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```

```
# %% [markdown]
# ## Score Dispersion per Topic
# %%
plt.figure(figsize=(10, 5))
plt.bar(
    score_std.keys(),
    score_std.values(),
    color="teal"
)
plt.title("Score Dispersion per Topic (Standard Deviation)")
plt.ylabel("Score standard deviation")
plt.xlabel("Topic")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# %% [markdown]
# ## Stress test plot
# %%
x = range(len(stress_df))
fig, ax1 = plt.subplots(figsize=(10, 5))
# Correlations (bars)
ax1.bar(x, stress_df["Spearman"], width=0.35, label="Spearman", color="steelblue")
ax1.bar(
    [i + 0.35 for i in x],
    stress_df["Kendall"],
    width=0.35,
    label="Kendall",
    color="orange"
)
ax1.set_ylabel("Correlation")
ax1.set_ylim(0, 1)
ax1.set_xticks([i + 0.175 for i in x])
ax1.set_xticklabels(stress_df["Scenario"], rotation=45, ha="right")
ax1.legend(loc="upper left")
# Flip rate (line, secondary axis)
ax2 = ax1.twinx()
ax2.plot(
    [i + 0.175 for i in x],
    stress_df["FlipRate"],
    color="red",
    marker="o",
    label="Flip Rate"
)
ax2.set_ylabel("Flip rate")
# Title
plt.title("Stress Test — Ranking Robustness (Correlation & Flip Rate)")
fig.tight_layout()
plt.show()

# %%
# %% [markdown]
# ## Structural limit tests: Alpha → 0, Beta → {0,1}
structural_results = []
for beta_test in [0.0, 1.0]:
    eta_beta = compute_eta_matrix(topic_df, beta_test)
    eta_beta = eta_beta.loc[eta_matrix.index]
    eta_prime_beta = compute_eta_prime(eta_beta, M)
    scores_beta = compute_scores(weights, eta_prime_beta)
    ranks_beta = scores_beta.rank(ascending=False)
    rho, _ = spearmanr(baseline_ranks, ranks_beta)
    tau, _ = kendalltau(baseline_ranks, ranks_beta)
    structural_results.append({
        "Scenario": f"Beta = {beta_test}",
        "Spearman": rho,
        "Kendall": tau
    })
# Alpha → 0 (no interdependency)
eta_prime_alpha0 = compute_eta_prime(eta_matrix, np.eye(M.shape[0]))
scores_alpha0 = compute_scores(weights, eta_prime_alpha0)
ranks_alpha0 = scores_alpha0.rank(ascending=False)
rho, _ = spearmanr(baseline_ranks, ranks_alpha0)
tau, _ = kendalltau(baseline_ranks, ranks_alpha0)
structural_results.append({
    "Scenario": "Alpha = 0",
    "Spearman": rho,
    "Kendall": tau
})
structural_df = pd.DataFrame(structural_results)
structural_df

# %%
```

```
# %% [markdown]
# ## Objective dominance tests (one objective → 1)
dominance_results = []
for obj in weights.keys():
    dominant_weights = {k: 0.0 for k in weights}
    dominant_weights[obj] = 1.0
    scores_dom = compute_scores(dominant_weights, eta_prime)
    ranks_dom = scores_dom.rank(ascending=False)
    rho, _ = spearmanr(baseline_ranks, ranks_dom)
    tau, _ = kendalltau(baseline_ranks, ranks_dom)
    dominance_results.append({
        "Dominant objective": obj,
        "Spearman": rho,
        "Kendall": tau
    })
dominance_df = pd.DataFrame(dominance_results)
dominance_df


# %%
# %% [markdown]
# ## Matrix conditioning & numerical stability
cond_M = np.linalg.cond(M)
print("Condition number of M:", cond_M)
if cond_M > 1e6:
    print("⚠️ Warning: M may be ill-conditioned.")
else:
    print("✅ M is well-conditioned.")
# Finite-value check
assert np.isfinite(eta_prime.values).all(), "Non-finite values detected in eta_prime"


# %%
# %% [markdown]
# ## Non-uniform uncertainty model (truncated Gaussian)
GAUSS_STD = UNCERTAINTY / 2
spearman_gauss = []
kendall_gauss = []
for _ in range(N_SIM):
    # Gaussian ORI perturbation
    perturbed_weights = {
        k: v * (1 + np.random.normal(0, GAUSS_STD))
        for k, v in weights.items()
    }
    total = sum(perturbed_weights.values())
    perturbed_weights = {k: max(v / total, 0) for k, v in perturbed_weights.items()}
    # Gaussian topic perturbation
    perturbed_topic_df = topic_df.copy()
    for col in ["r_Av","r_Phy","r_AM","r_CIA","u_Av","u_Phy","u_AM","u_CIA","v_j"]:
        perturbed_topic_df[col] *= (1 + np.random.normal(0, GAUSS_STD))
        perturbed_topic_df[col] = perturbed_topic_df[col].clip(0, 1)
    eta_mc = compute_eta_matrix(perturbed_topic_df, BETA)
    eta_mc = eta_mc.loc[eta_matrix.index]
    eta_prime_mc = compute_eta_prime(eta_mc, M)
    scores_mc = compute_scores(perturbed_weights, eta_prime_mc)
    ranks_mc = scores_mc.rank(ascending=False)
    rho, _ = spearmanr(baseline_ranks, ranks_mc)
    tau, _ = kendalltau(baseline_ranks, ranks_mc)
    spearman_gauss.append(rho)
    kendall_gauss.append(tau)
print("Gaussian noise Spearman mean:", np.mean(spearman_gauss))
print("Gaussian noise Kendall mean :", np.mean(kendall_gauss))


# %%
# %% [markdown]
# ## Rank displacement metrics (expected deviation)
rank_displacement = {topic: [] for topic in baseline_ranks.index}
for _ in range(N_SIM):
    perturbed_weights = {k: v * (1 + np.random.uniform(-UNCERTAINTY, UNCERTAINTY)) for k, v in weights.items()}
    total = sum(perturbed_weights.values())
    perturbed_weights = {k: v / total for k, v in perturbed_weights.items()}
    perturbed_topic_df = topic_df.copy()
    for col in ["r_Av","r_Phy","r_AM","r_CIA","u_Av","u_Phy","u_AM","u_CIA","v_j"]:
        perturbed_topic_df[col] *= (1 + np.random.uniform(-UNCERTAINTY, UNCERTAINTY))
        perturbed_topic_df[col] = perturbed_topic_df[col].clip(0, 1)
    eta_mc = compute_eta_matrix(perturbed_topic_df, BETA)
    eta_mc = eta_mc.loc[eta_matrix.index]
    eta_prime_mc = compute_eta_prime(eta_mc, M)
    scores_mc = compute_scores(perturbed_weights, eta_prime_mc)
    ranks_mc = scores_mc.rank(ascending=False)
    for topic in baseline_ranks.index:
        rank_displacement[topic].append(
            abs(ranks_mc.loc[topic] - baseline_ranks.loc[topic])
        )
expected_displacement = {
    topic: np.mean(vals)
```

```
    for topic, vals in rank_displacement.items()
}
disp_df = pd.DataFrame.from_dict(
    expected_displacement,
    orient="index",
    columns=["Expected rank displacement"]
).sort_values("Expected rank displacement", ascending=False)
disp_df

# %%
# %% [markdown]
# ## Structural limit tests — correlation comparison
plt.figure(figsize=(8, 4))
plt.bar(
    structural_df["Scenario"],
    structural_df["Spearman"],
    label="Spearman",
    color="steelblue"
)
plt.bar(
    structural_df["Scenario"],
    structural_df["Kendall"],
    label="Kendall",
    color="orange",
    alpha=0.7
)
plt.ylim(0, 1)
plt.ylabel("Correlation with baseline")
plt.title("Structural Limit Tests — Ranking Consistency")
plt.legend()
plt.xticks(rotation=30)
plt.tight_layout()
plt.show()

# %%
# %% [markdown]
# ## Objective dominance — correlation chart
plt.figure(figsize=(8, 4))
plt.bar(
    dominance_df["Dominant objective"],
    dominance_df["Spearman"],
    label="Spearman",
    color="purple"
)
plt.bar(
    dominance_df["Dominant objective"],
    dominance_df["Kendall"],
    label="Kendall",
    color="gold",
    alpha=0.7
)
plt.ylim(0, 1)
plt.ylabel("Correlation with baseline")
plt.title("Objective Dominance Test")
plt.legend()
plt.tight_layout()
plt.show()

# %%
# %% [markdown]
# ## Matrix conditioning diagnostic
plt.figure(figsize=(5, 3))
plt.bar(["cond(M)"], [cond_M], color="red" if cond_M > 1e6 else "green")
plt.axhline(1e6, linestyle="--", color="black", label="Ill-conditioning threshold")
plt.yscale("log")
plt.ylabel("Condition number (log scale)")
plt.title("Interdependency Matrix Conditioning")
plt.legend()
plt.tight_layout()
plt.show()

# %%
# %% [markdown]
# ## Noise model comparison — Spearman distributions
plt.figure(figsize=(8, 4))
plt.hist(spearman_vals, bins=30, alpha=0.6, label="Uniform noise")
plt.hist(spearman_gauss, bins=30, alpha=0.6, label="Gaussian noise")
plt.xlabel("Spearman ρ")
plt.ylabel("Frequency")
plt.title("Noise Model Comparison — Ranking Robustness")
plt.legend()
plt.tight_layout()
plt.show()
```

```
# %%
# %% [markdown]
# ## Expected rank displacement per topic
plt.figure(figsize=(10, 5))
plt.bar(
    disp_df.index,
    disp_df["Expected rank displacement"],
    color="teal"
)
plt.ylabel("Expected rank displacement")
plt.title("Topic-Level Ranking Stability")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

# %%
# %% [markdown]
# ## Global robustness dashboard
fig, axs = plt.subplots(2, 2, figsize=(12, 8))
axs[0, 0].hist(spearman_vals, bins=30, color="steelblue")
axs[0, 0].set_title("Spearman (Uniform)")
axs[0, 1].hist(spearman_gauss, bins=30, color="orange")
axs[0, 1].set_title("Spearman (Gaussian)")
axs[1, 0].bar(dominance_df["Dominant objective"], dominance_df["Spearman"])
axs[1, 0].set_title("Objective Dominance")
axs[1, 1].bar(disp_df.index, disp_df["Expected rank displacement"])
axs[1, 1].set_title("Rank Displacement")
for ax in axs.flat:
    ax.tick_params(axis="x", rotation=30)
plt.suptitle("Global Ranking Robustness Summary")
plt.tight_layout()
plt.show()
```